

Chapter Five

The Transport Layer

Objectives

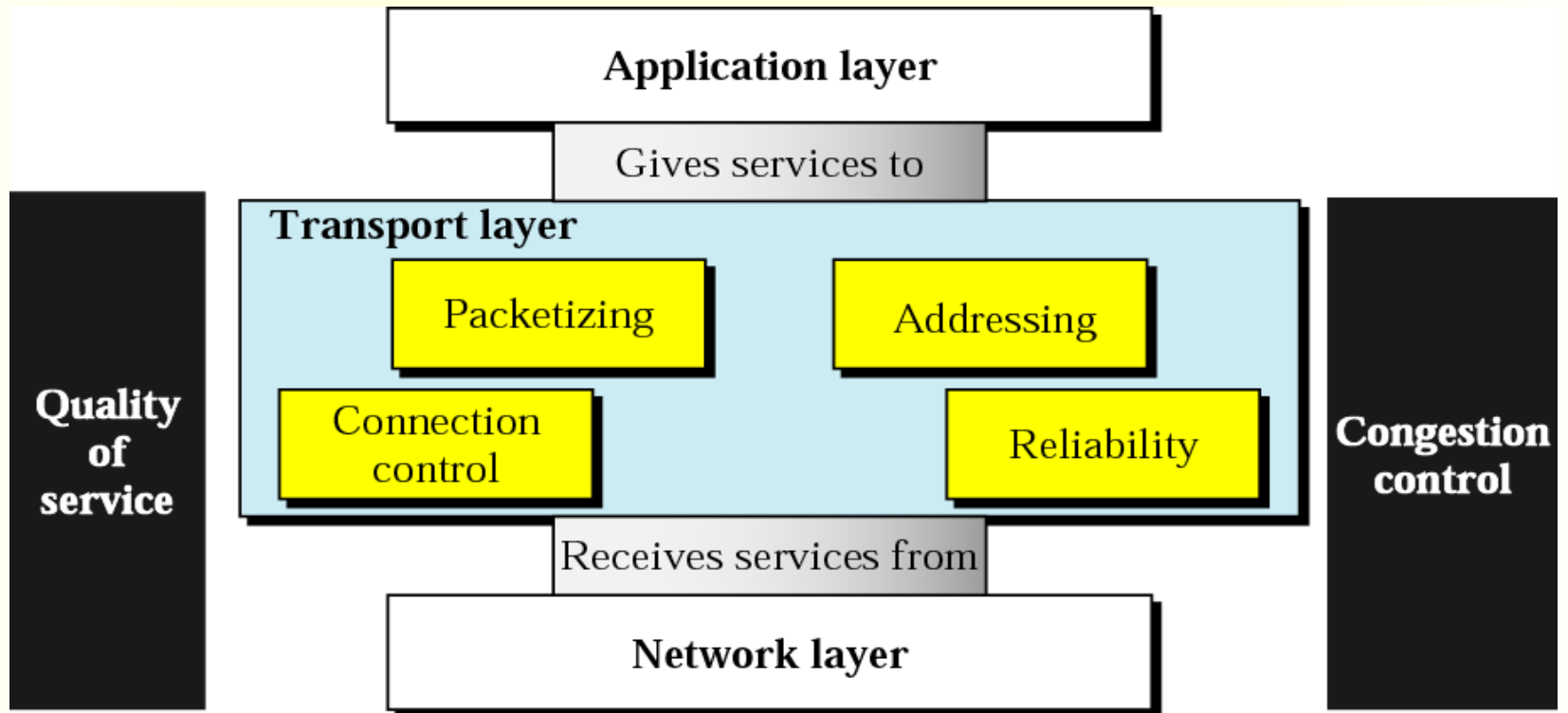
- The transport layer is responsible for process-to-process delivery of the entire message.
- A process is an application program running on a host.
- Whereas the network layer oversees source-to-destination delivery of individual packets, it does not recognize any relationship between those packets.
- It treats each one independently, as though each piece belonged to a separate message, whether or not it does.
- The transport layer ensures that the whole message arrives intact and in order, overseeing both error control and flow control at the source-to-destination level.

3.1 Introduction

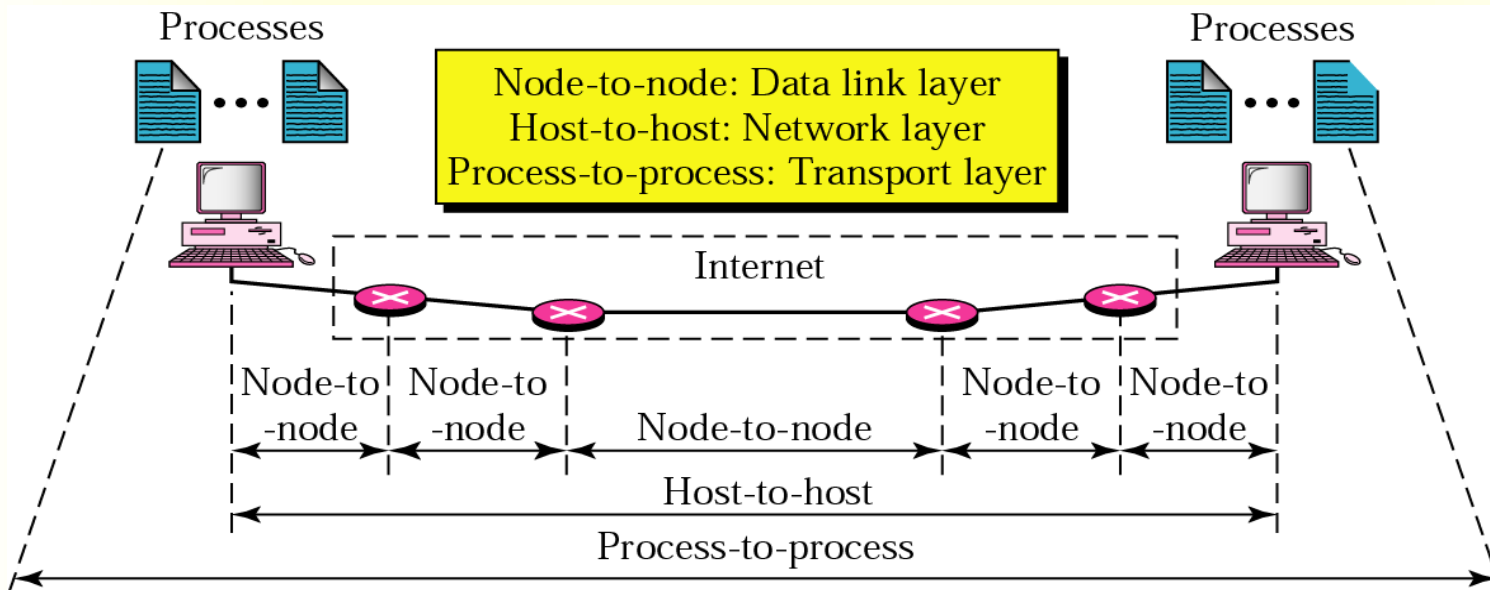
- the core or the heart of the whole protocol hierarchy
- it provides **reliable, cost-effective** data transfer from the source to the destination on top of an unreliable network; **process-to-process** delivery
- the transport layer code runs on the users' machines; the network layer mostly runs on the routers (operated by a carrier for a WAN) and the adequacy of the service can not be guaranteed
- position of the transport layer

Cont'd...

- **A transport layer protocol can be either connectionless or connection-oriented**
- **A connectionless transport layer treats each segment as an independent packet and delivers it to the transport layer at the destination machine.**
- **A connection-oriented transport layer makes a connection with the transport layer at the destination machine first before delivering the packets. After all the data is transferred, the connection is terminated**



- data link layer: node-to-node delivery
- network layer: host-to-host delivery
- transport layer: process-to-process delivery



- several processes may be running on the source as well as the destination; an addressing mechanism is required (see next slide)

- two processes communicate in a **client-server** relationship (paradigm)
 - a process on the local host, called a **client**, needs services from a process usually on the remote host, called a **server**
 - both processes (client and server) have the same name; e.g., to get the day and time from a remote machine, we need a **Daytime** client process running on the local host and a **Daytime** server process running on a remote machine
 - for communication, we must define (identify them by an addressing mechanism) the following four
 - Local Host
 - Local Process
 - Remote Host
 - Remote Process

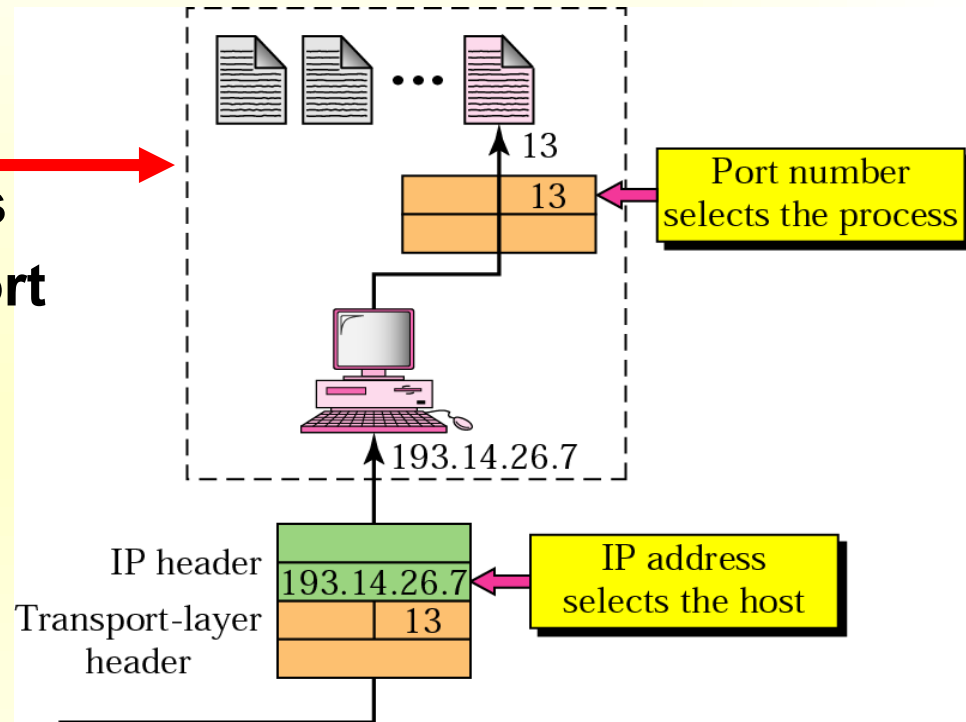
3.2 Packetizing

- the transport layer creates packets out of the message received from the application layer
- it divides a long message into smaller ones; they are then encapsulated into the data field of the transport-layer packet and headers are added

3.3 Addressing

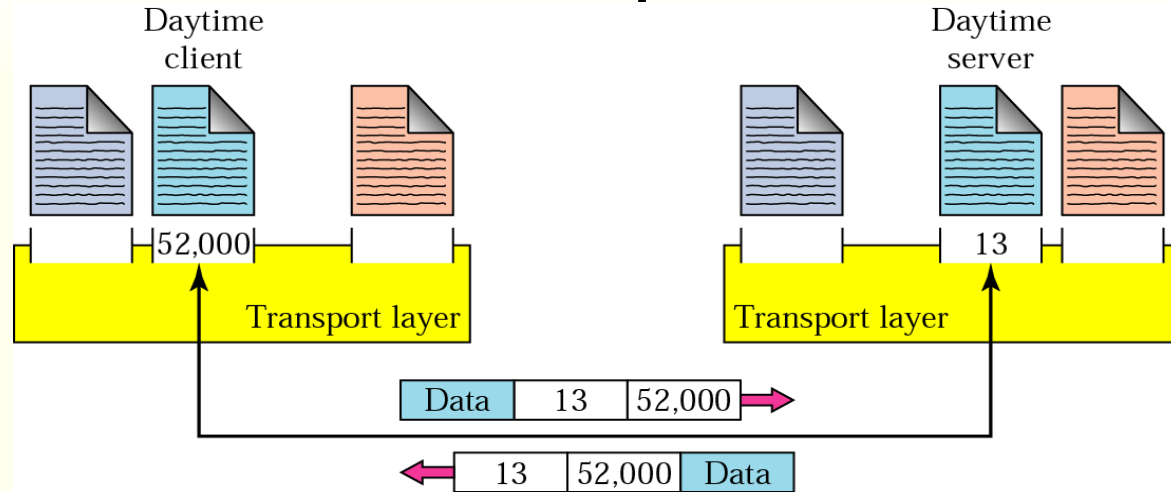
- at the **data link** layer, we need a **MAC** address
- at the **network** layer, we need an **IP** address
- at the **transport** layer, we need a transport-layer address, called a **port number**, to choose among several processes

- in all the above, a destination address is needed for delivery and a source address for reply (next **node**, destination **host**, and destination **process**, respectively)
- IP addresses and port numbers play different roles
- in the Internet model, the port numbers are **16-bit** integers between 0 and 65,535

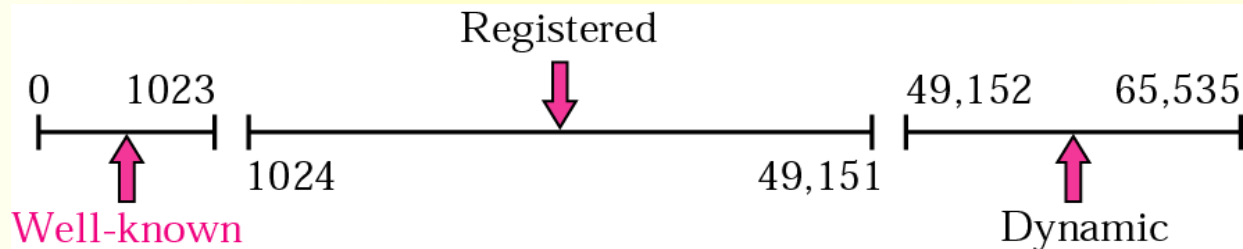


- **ephemeral port number**: the client program defines itself with a port number, chosen randomly by the transport layer software running on the client

- **well-known port numbers**: universal port numbers for servers; not random since a client process cannot know it



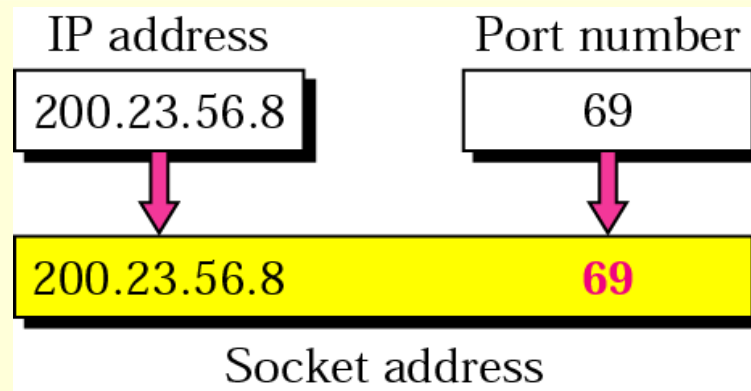
- **IANA (Internet Assigned Numbers Authority) Ranges**
 - IANA divided the port numbers into three ranges



- **Well-known ports**: assigned and controlled by IANA for standard services
- **Registered ports**: are not assigned and controlled by IANA; can only be registered with IANA to prevent duplication
- **Dynamic ports**: neither controlled nor registered; these are the **ephemeral ports**

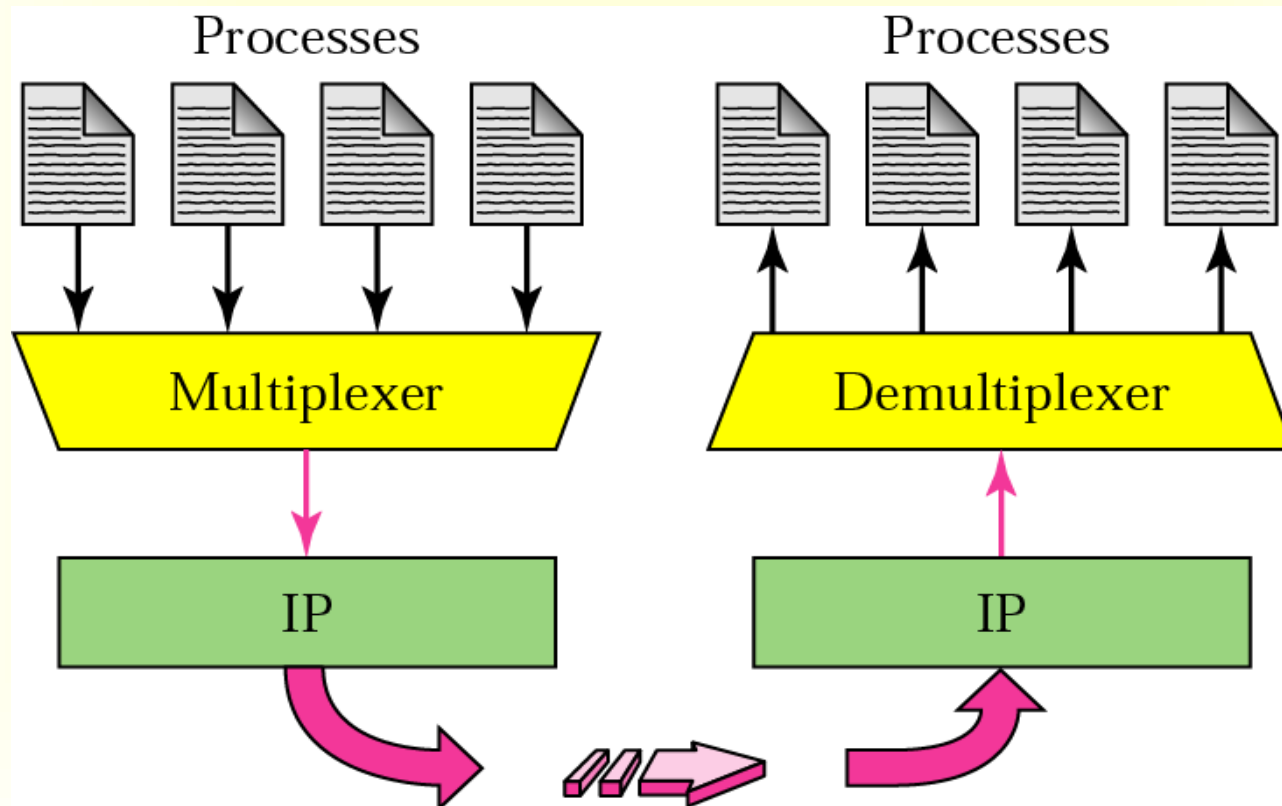
■ Socket Addresses

- process-to-process delivery needs two addresses: IP address and port number at each end
- the combination of an IP address and a port number is called a **socket address**



- a transport-layer protocol needs a pair of socket addresses: the client socket address and the server socket address
- the IP header contains the IP address; the UDP or TCP header contains the port number

- **Multiplexing and Demultiplexing**
 - the addressing mechanism allows multiplexing and demultiplexing by the transport layer since there may be several processes that need to send packets, but there is only one transport-layer protocol (UDP or TCP)



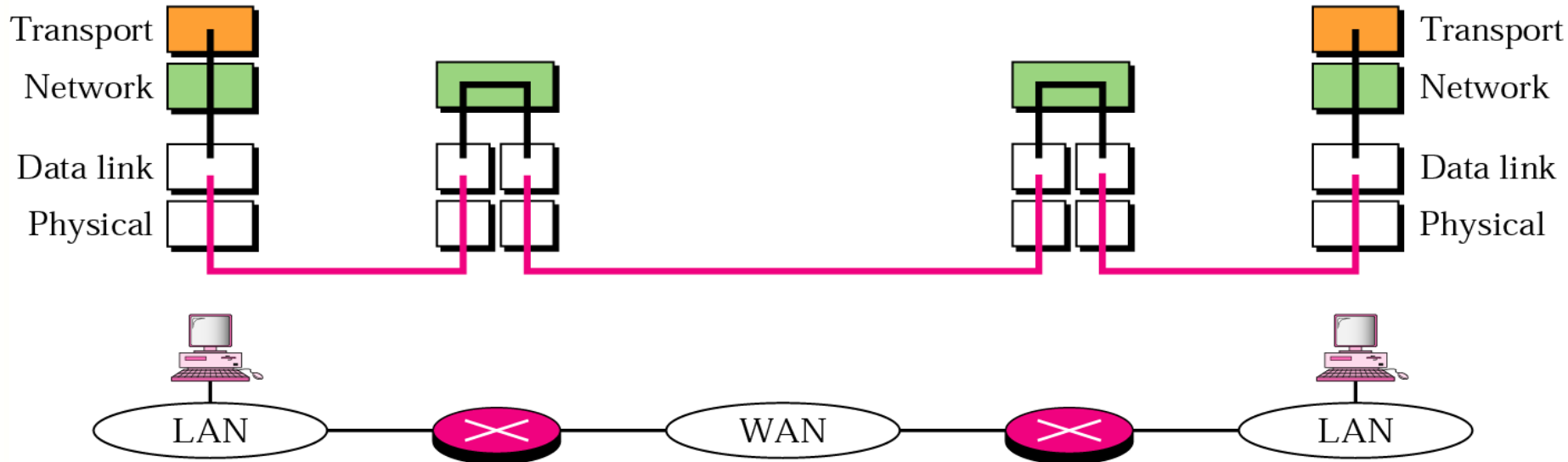
3.4 Connection Control

- a transport-layer protocol can be either **connectionless** (UDP) or **connection-oriented** (TCP)
- if connection-oriented, it first establishes a virtual connection, then follows data transfer, then the connection is terminated

3.5 Reliability - UDP and TCP

- a transport-layer service can be either **reliable** or **unreliable**
- the choice depends on the needs of the application layer program
- a reliable transport-layer protocol (TCP) implements **flow** and **error control**; a **slower** and more **complex** service
- if the data link layer is reliable and has flow and error control, do we need this at the transport layer?
- yes; reliability at the data link layer is between two nodes, but we also need reliability between two ends
- flow control at the transport layer is end-to-end rather than across a single link

— Error is checked in these paths by the data link layer
— Error is not checked in these paths by the data link layer



- **reliability may not be needed (UDP)**
 - a process may use its own flow and error control mechanisms
 - a process may need fast service
 - the nature of the service may not demand flow and error control (real-time applications)

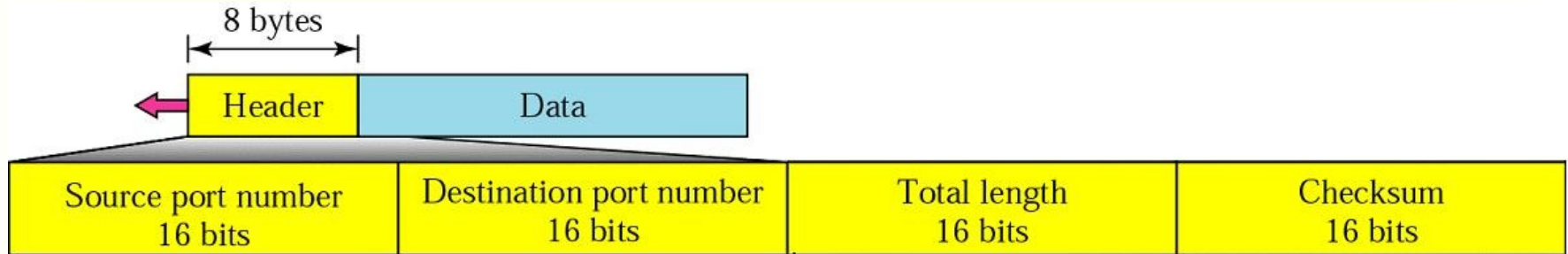
■ **UDP - User Datagram Protocol**

- **connectionless** and **unreliable** transport layer protocol in the Internet (no flow or error control, no retransmission if data is corrupted or lost)
- **simple with a minimum of overhead**
- **convenient for**
 - **multimedia and multicasting applications**
 - **for client-server situations**
 - **e.g., DNS can use UDP, give me the IP address of the host name `www.cs.aau.edu.et` - see later**

- some of the well-known ports used by UDP (see www.iana.org for a complete list - also for TCP ports)

Port	Protocol	Description
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
53	DNS	Domain Name Service
67	Bootps	Server port to download bootstrap information
68	Bootpc	Client port to download bootstrap information
69	TFTP	Trivial File Transfer Protocol
111	RPC	Remote Procedure Call
123	NTP	Network Time Protocol
161	SNMP	Simple Network Management Protocol
162	SNMP	Simple Network Management Protocol (trap)

- UDP packets, called datagrams, have a fixed size header of 8 bytes followed by the payload (data)



- Checksum: optional; if not included, the field is filled with 0s (a true computed 0 is stored as all 1s)
- **Some Applications of UDP**
 - for processes that require simple **request-response** communication; not for applications that send bulk data such as FTP
 - for a process with **internal** flow and error control mechanisms, e.g., TFTP (Trivial FTP)

- for **multicasting**; multicasting capabilities are embedded in the UDP software but not in the TCP software
- used for some route updating protocols such as RIP (Routing Information Protocol)
- used in conjunction with **RTP** (Real Time Transport Protocol) to provide a transport-layer mechanism for real-time data; see later for RTP in the discussion of multimedia
- **TCP - Transport Control Protocol**
 - reliable, but complex transport-layer protocol in the Internet
 - it is called a **stream connection-oriented** and **reliable** transport protocol
 - like UDP, TCP uses port numbers as transport-layer addresses
 - note: if an application can use both UDP and TCP, the same port number is assigned to this application; e.g., Daytime, DNS

■ some of the well-known ports used by TCP

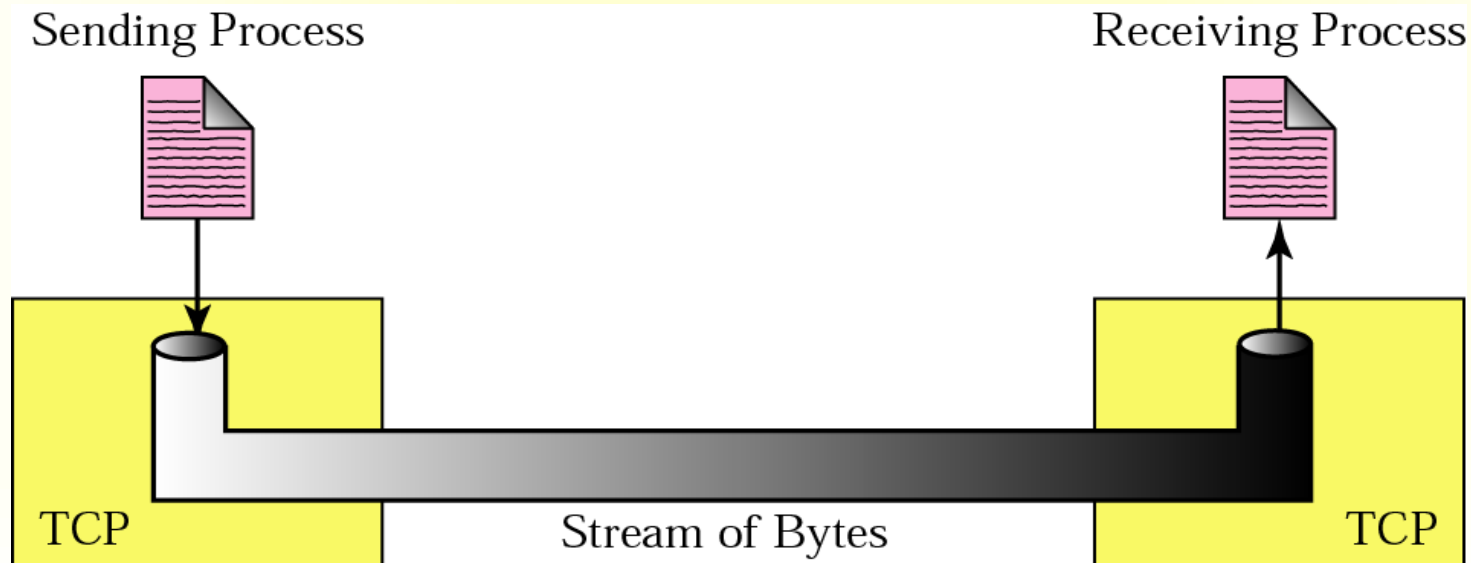
Port	Protocol	Description
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
20	FTP, Data	File Transfer Protocol (data connection)
21	FTP, Control	File Transfer Protocol (control connection)
23	TELNET	Terminal Network
25	SMTP	Simple Mail Transfer Protocol
53	DNS	Domain Name Server
67	BOOTP	Bootstrap Protocol
79	Finger	Lookup information about a user
80	HTTP	Hypertext Transfer Protocol
111	RPC	Remote Procedure Call

- **Important issues (topics) to be covered for TCP**
 - **TCP Services**
 - **Numbering Bytes**
 - **Segments and the TCP header format**
 - **Flow Control**
 - **Error Control**

■ TCP Services

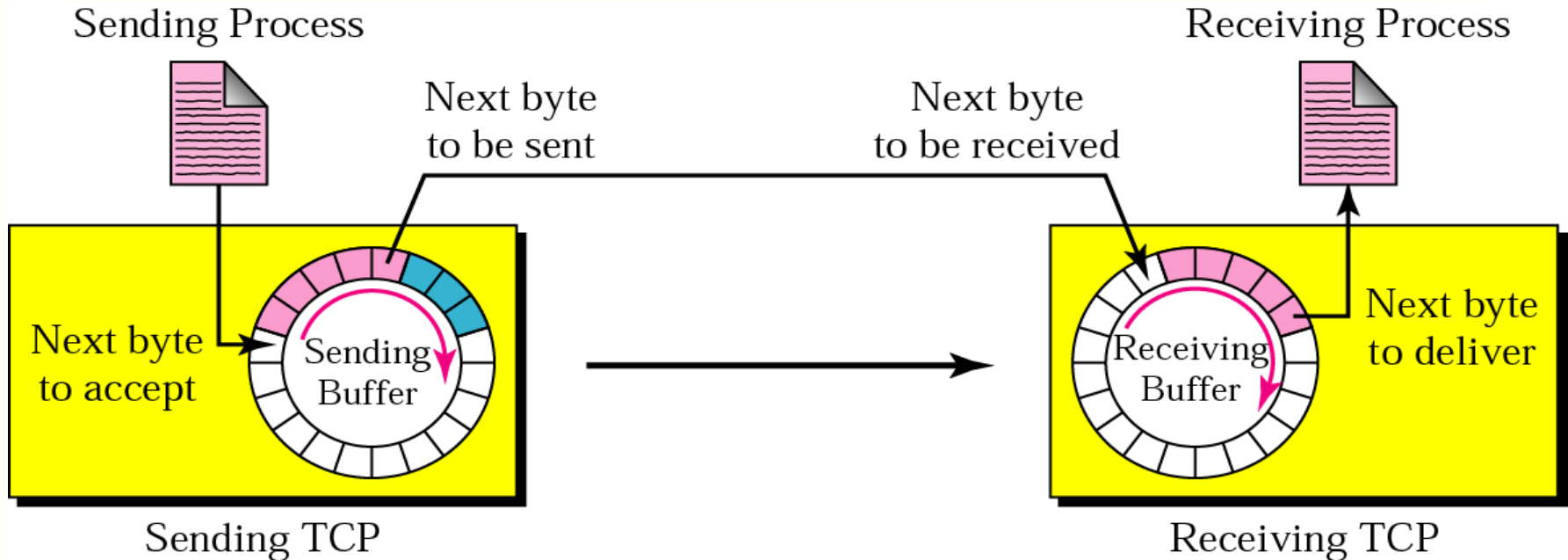
1. Stream Delivery Service

- unlike UDP, TCP is a stream-oriented protocol; in UDP, there are no relationships between different chunks of data
- in TCP, the two processes seem to be connected by an imaginary “tube” that carries their data across the Internet



2. Sending and Receiving Buffers

- the sender and the receiver may not produce and consume data at the same speed

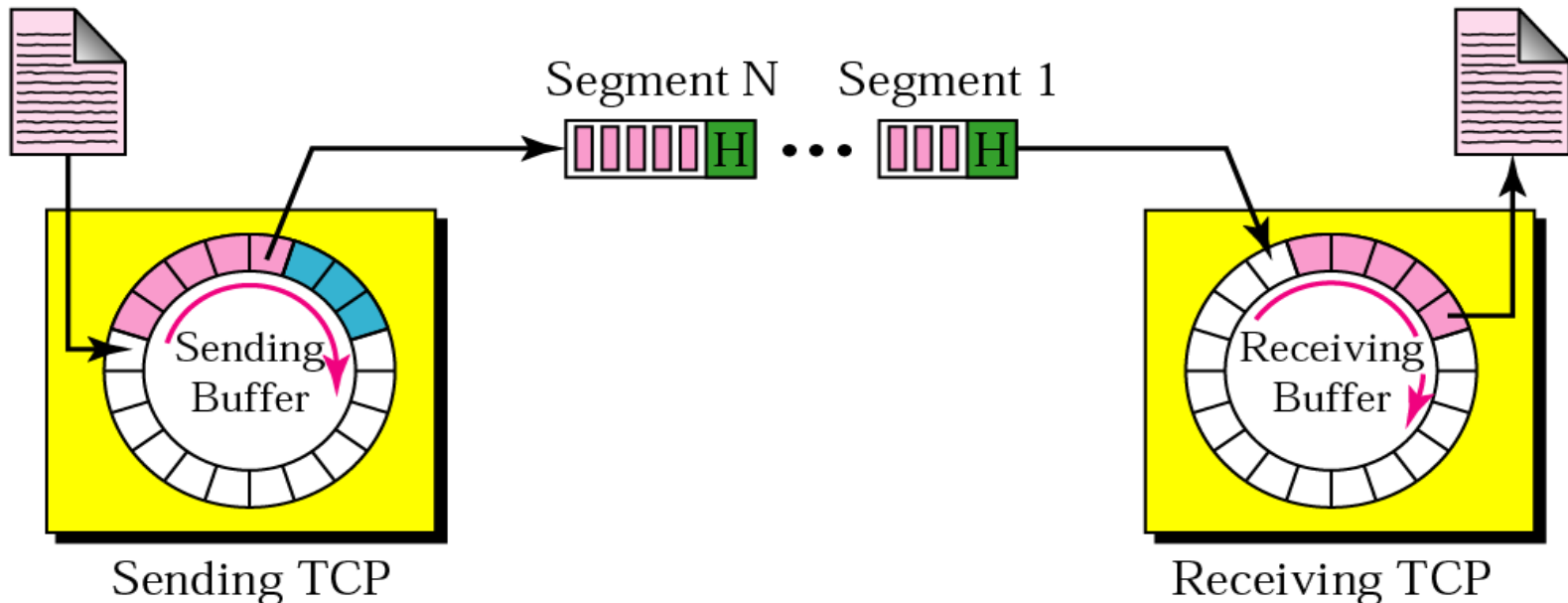


3. Bytes and Segments

- the IP layer (as a service provider for TCP) needs to send data in packets, not as a stream of bytes
- TCP groups a number of bytes together into a packet called a **segment**; it adds a header to each segment (for control purposes) and delivers it to IP

Sending Process

Receiving Process



4. Full-Duplex Service

- TCP offers a full-duplex service
- each TCP has a sending and receiving buffer, and segments are sent in both directions

5. Connection-Oriented Service

- a virtual connection for all segments belonging to a message is created
- a TCP segment is encapsulated in an IP datagram and can be sent out of order, lost, or corrupted, and then resent
- however, TCP creates a stream-oriented environment in which it accepts the responsibility of delivering the bytes in order to the other site
- the connection is point-to-point (it has exactly two ends); TCP does not support **multicasting** or **broadcasting**

6. Reliable Service

- it uses an acknowledgement mechanism to check the safe arrival of data (see later in error control)

■ Numbering Bytes

- there are two fields in the TCP header: **sequence number** and **acknowledgement number**; they refer to the **byte number** not the **segment number**

■ Byte Numbers

- TCP numbers all data bytes transmitted; numbering is independent in each direction; used for flow and error control
- the numbering starts with a randomly generated number (between 0 and $2^{32}-1$)

■ Sequence Number

- after the bytes have been numbered, TCP assigns a sequence number to each segment
- the sequence number is the number of the first byte carried in that segment

- e.g., Imagine a TCP connection is transferring a file of 6000 bytes. The first byte is numbered 10,010. What are the sequence numbers for each segment if data are sent in five segments with the first four segments carrying 1000 bytes and the last segment carrying 2000 bytes?

The following shows the sequence number for each segment:

Segment 1 ==> sequence number: 10,010 (range: 10,010 to 11,009)

Segment 2 ==> sequence number: 11,010 (range: 11,010 to 12,009)

Segment 3 ==> sequence number: 12,010 (range: 12,010 to 13,009)

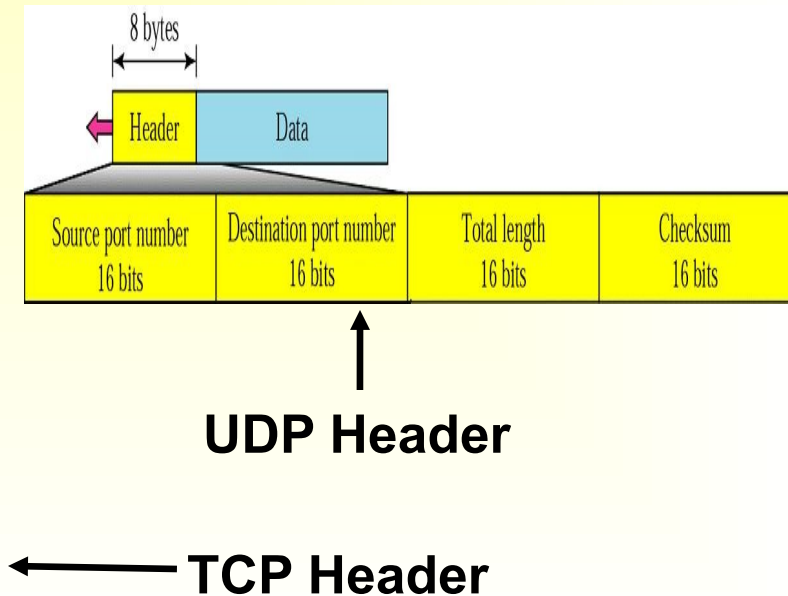
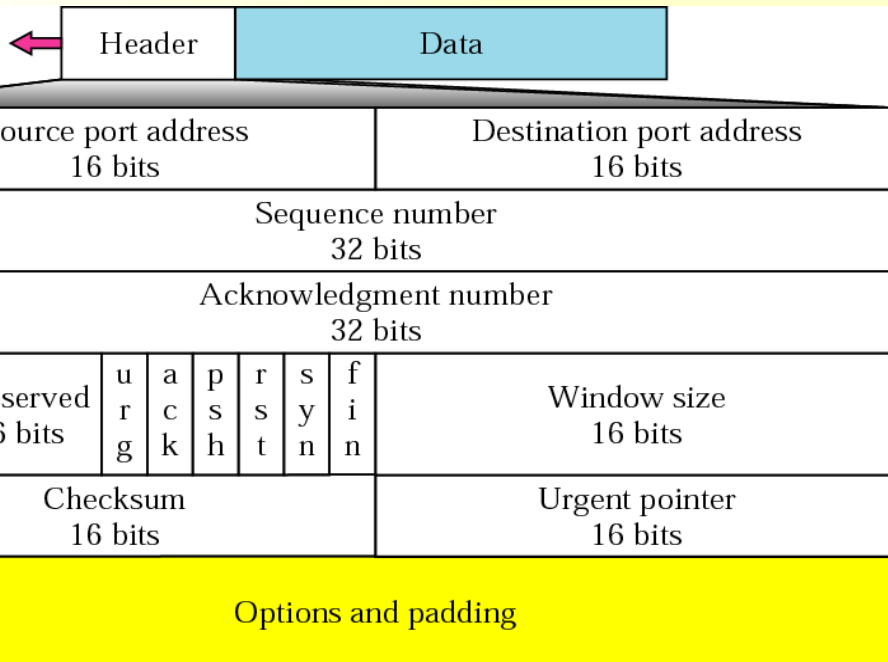
Segment 4 ==> sequence number: 13,010 (range: 13,010 to 14,009)

Segment 5 ==> sequence number: 14,010 (range: 14,010 to 16,009)

- **Acknowledgement Number**
 - to confirm the bytes the receiver has received
 - it defines the number of the next byte that the receiver **expects** to receive (both can be receivers since the transmission is full-duplex)
 - it is **cumulative**; the receiver adds 1 to the number of the last byte it received correctly and announces this as the acknowledgement number
 - e.g., if a party uses 5643 as an acknowledgement number, it means that it has received all bytes from the beginning up to 5642 (does it mean it has received 5642 bytes?)

■ Segments

- the unit of data transfer using TCP is a segment
- a segment consists of a 20- to 60-byte header followed by data
- **Header length (HLEN)**: the number of 4-byte words in the TCP header (because of the options)
- **Control**: 6 different control bits or flags; they enable flow control, connection establishment and termination, and the mode of data transfer in TCP



Flag Description

URG	The value of the urgent pointer field is valid
ACK	The value of the acknowledgment number field is valid
PSH	Push the data; send it without waiting for the window to be filled
RST	Connection must be reset - as a result of host crash or other reasons
SYN	Synchronize sequence numbers during connection establishment
FIN	Terminate the connection

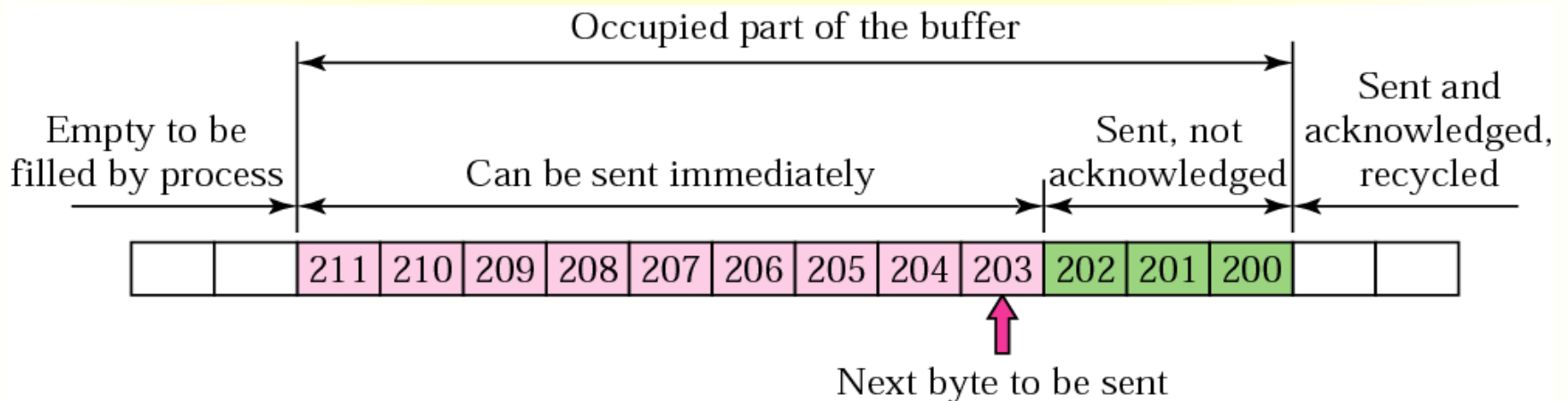
- **Urgent pointer:** the segment contains urgent data; it must be delivered soon even if out of order, e.g., Control-C to abort the application program on the other side; valid only if the urgent flag is set
- **Widow size:** defines the size of the sliding window, in bytes, that the other party must maintain

■ Flow Control

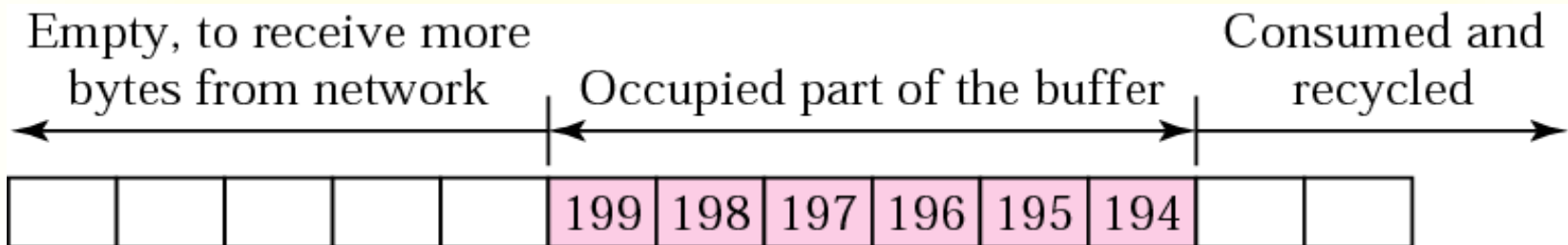
- defines the amount of data a source can send before receiving an acknowledgement from the destination; used to prevent a fast transmitter from overwhelming a slow receiver
- two extreme cases
 - send 1 byte: extremely slow
 - send all the data: speeds up the process; but may overwhelm the receiver; + if some part of data is lost, duplicated, received out of order, or corrupted, the source will not know until all data have been checked by the destination
- TCP defines a window (somewhere in between the above two extremes)
- it sends as many data as are defined by the sliding window protocol

■ Sliding Window Protocol

- both hosts use a window for each connection
- used to make transmission more efficient as well as to control the flow of data so that the destination does not become overwhelmed
- TCP's sliding windows are byte-oriented
- sender buffer (circular)



■ receiver buffer (circular)

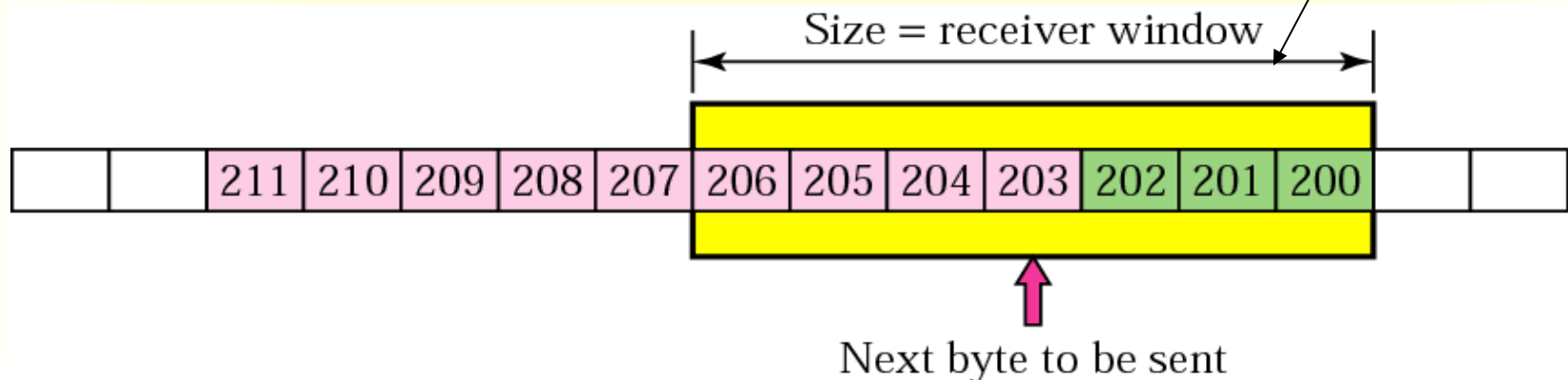


■ Receiver window

- how many more bytes can the receiver store?
- let N = the total size of the receiving buffer
 M = the number of locations occupied
- then only $N-M$ more bytes can be received; this is called the **receiver window**; in the above example it is $13-6 = 7$

■ Sender Window

- we have flow control if the sender creates a window (the sender window) with a size less than or equal to the size of the receiver window
- sender buffer and sender window



■ Error Control

- error control in TCP includes mechanisms for detecting corrupted segments (discarded by the destination), lost segments (discarded by some intermediate node), out-of-order segments, and duplicated segments
- most errors nowadays are caused by congestion rather than error-prone transmission media
- it uses three simple tools: **checksum**, **acknowledgement**, and **time-out**
- no **negative acknowledgement** is used in TCP; if a segment is not acknowledged before the time-out, it is considered to be either corrupted or lost

3.6 Congestion Control

- they are related to three layers: data link, network, and transport layers
- congestion: may occur if the **load** on the network is greater than the **capacity** of the network
- it arises as a result of waiting (queues by routers)
- when a packet is delayed, the source, not receiving the acknowledgement, retransmits the packet, which makes the delay, and the congestion, worse
- congestion control: refers to the mechanisms and techniques to control the congestion and keep the load below the capacity; either prevent congestion before it happens or remove congestion after it has happened (you can read more on the available techniques)